dbt Cheatsheet

Sessions 1–7

One-page reference for the Session 7 midterm.

CLI commands

Command	Purpose
`dbt debug`	Verify connection + project config
`dbt deps`	Install packages from `packages.yml`
`dbt seed`	Load `seeds/*.csv` as tables
`dbt compile`	Render Jinja → SQL, no execution. Output: `target/compiled/`
`dbt run`	Execute all models in DAG order. Output: `target/run/`
`dbt test`	Run only tests
`dbt build`	**`run + test + seed + snapshot`** in DAG order. **Prefer this.**
`dbt source freshness`	Check `loaded_at_field` against thresholds
`dbt docs generate`	Build `catalog.json` for docs site
`dbt docs serve`	Serve docs + lineage at `localhost:8080`
`dbt ls`	List nodes (models, tests, seeds…)
`dbt run-operation <macro>`	Execute a macro outside model context

Flags: `--profiles-dir`, `--project-dir`, `--target <name>`, `--full-refresh`

Selector syntax (`-s`)

Selector	Meaning
`model_name`	just this model
`+model_name`	this + **upstream** (ancestors)
`model_name+`	this + **downstream** (descendants)
`+model_name+`	this + both directions
`2+model_name`	this + 2 levels upstream only
`model_name+3`	this + 3 levels downstream only
`staging`	everything in `models/staging/` folder
`tag:finance`	everything with tag "finance"
`path:models/marts`	by file path
`source:raw.customers`	a source node
`--exclude tag:slow`	inverse — drop matches

Project structure

```
dbt-ie/
├── dbt_project.yml         # project config (name, paths, defaults)
├── profiles.yml            # connection (adapter, db path, schema)
├── packages.yml            # external dbt packages
├── models/                 # transformations (.sql + .py + .yml)
│   ├── staging/            # one stg_ per source, views
│   ├── intermediate/       # joins/logic, internal use
│   ├── marts/              # dim_ + mart_, tables
│   └── sources.yml         # source definitions
├── seeds/                  # static CSVs → tables
├── macros/                 # reusable Jinja
├── snapshots/              # SCD Type 2
├── tests/                  # singular/custom tests
├── analyses/               # ad-hoc SQL (NOT run by dbt run)
├── target/                 # compiled + run artifacts (gitignored!)
└── logs/                   # dbt logs (gitignored)
```

`profiles.yml` resolution (first match wins)

1. `--profiles-dir /path` CLI flag
2. `DBT_PROFILES_DIR` env var
3. **Project directory** (next to `dbt_project.yml`)
4. `~/.dbt/profiles.yml`

```yaml
default:
  target: dev
  outputs:
    dev: {type: duckdb, path: my_database.duckdb, schema: main}
```

`dbt_project.yml` essentials

```yaml
name: 'dbt_ie'
profile: 'default'           # which profile to use
model-paths: ["models"]
seed-paths:  ["seeds"]
models:
  dbt_ie:
    staging:      {+materialized: view}
    intermediate: {+materialized: ephemeral}
    marts:        {+materialized: table}
    +grants: {select: ['reporter', 'analyst']}
```

`ref()` vs `source()`

Function	When	Defined in
`{{ source('raw', 'customers') }}`	Reading a **raw** warehouse table	`sources.yml`
`{{ ref('stg_customers') }}`	Reading a **dbt-built model**	the `.sql` file's path
`{{ ref('segments') }}`	Reading a **seed**	`seeds/segments.csv`

Both build the DAG. Hardcoded names (`from main.customers`) **break lineage** — never do it.

Materializations

Type	DDL	Build	Query	Default for
`view`	`CREATE VIEW`	Fast	Slow	**staging**
`table`	`CREATE TABLE AS SELECT`	Slow	Fast	**marts**
`ephemeral`	None — inlined as CTE	n/a	n/a	sometimes intermediate
`incremental`	Update existing table	Fast on big data	Fast	big facts

Setting materialization

Project-level (`dbt_project.yml`):

```yaml
models:
  dbt_ie:
    staging: {+materialized: view}
```

Model-level (top of `.sql` file):

```
{{ config(materialized='table') }}
```

Model-level overrides project-level.

Generic tests (built-in)

Test	Catches
`unique`	Duplicates
`not_null`	NULLs
`accepted_values`	Values outside allowed set
`relationships`	Foreign-key integrity to another model

Tests in YAML

```yaml
version: 2
models:
  - name: stg_customers
    columns:
      - name: customer_id
        tests: [unique, not_null]
      - name: country
        tests:
          - accepted_values:
              values: [France, Spain, Sweden]
      - name: segment_id
        tests:
          - relationships:
              to: ref('segments')
              field: segment_id
```

Jinja essentials

Syntax	Meaning
`{{ ... }}`	Expression — output a value
`{% ... %}`	Statement — control flow / set / for / if
`{# ... #}`	Comment (not output)
`{% set x = 10 %}`	Variable assignment
`{% if cond %} ... {% endif %}`	Conditional
`{% for item in list %} ... {% endfor %}`	Loop
`loop.first`, `loop.last`, `loop.index`	Loop state
`{{ target.name }}`	The current target ("dev", "prod"...)
`{{ config(materialized='table') }}`	Model config

Jinja patterns

Trailing-comma trick (in a loop):

```
select
{% for col in cols %}
    {{ col }}{% if not loop.last %},{% endif %}
{% endfor %}
from {{ ref('my_model') }}
```

Target-aware filter (limit data in dev only):

```
select * from {{ ref('stg_orders') }}
{% if target.name == 'dev' %}
  where order_date > '2024-01-01'
{% endif %}
```

Macros

```
-- macros/cents_to_dollars.sql
{% macro cents_to_dollars(column_name) %}
    ({{ column_name }} / 100)::numeric(16, 2)
{% endmacro %}
```

Use in a model:

```
select {{ cents_to_dollars('amount_cents') }}
from {{ ref('stg_payments') }}
```

Run a macro outside a model:

```
dbt run-operation <macro_name> --args '{"key": "value"}'
```

Packages

```yaml
# packages.yml
packages:
  - package: dbt-labs/dbt_utils
    version: 1.1.1
  - package: dbt-labs/codegen
    version: 0.12.1
  - package: calogica/dbt_expectations
    version: 0.10.1
```

Install: `dbt deps`

Package	Highlights
dbt_utils	`generate_surrogate_key`, `union_relations`, `pivot`, `unpivot`
codegen	`generate_source`, `generate_base_model`, `generate_model_yaml`
dbt_expectations	`expect_column_values_to_be_between`, `..._to_be_of_type`

`target/` contents (after a build)

Path	What
`target/compiled/<proj>/models/...`	Jinja → SQL, no DDL
`target/run/<proj>/models/...`	Wrapped in materialization DDL
`target/manifest.json`	Full parsed project metadata
`target/run_results.json`	Last invocation result (timings, statuses)
`target/catalog.json`	Column-level docs (after `dbt docs generate`)

All disposable. Always in `.gitignore`.

Layer responsibilities

Layer	Prefix	Materialization	Job
Source	—	—	Raw data in warehouse
Seed	—	table	Static CSV lookups
Staging	`stg_`	view	1:1 with source, clean/rename/cast
Intermediate	`int_`	ephemeral or view	Joins, calculated fields, dedup
Marts	`dim_` / `mart_`	table	Business-ready facts/dims

SQL patterns by layer

Pattern	Where	Example
Cleaning & casting	Staging	`trim(lower(email))`, `::date`
`case when` classification	Intermediate	`case when total > 500 then 'high'`
Joins	Intermediate	`from a left join b using (id)`
Aggregation	Marts	`group by segment, month`
Window functions	Intermediate / Marts	`row_number() over (partition by …)`

Git workflow

1. `git checkout -b feature/<thing>` (branch)
2. Make **atomic commits** — one logical change each
3. `git push origin feature/<thing>` (push)
4. Open a **Pull Request** for review
5. **Merge** after approval

Common gotchas (exam-favorite traps)

- **`ref()` on a seed works** — seeds are nodes, use `ref('segments')` not `source(...)`
- **`target/compiled/` vs `target/run/`** — compiled = Jinja resolved, run = DDL-wrapped
- **Model-level `config()` always wins** over project-level in `dbt_project.yml`
- **`dbt run` does NOT run tests** — use `dbt build` for both
- **A failed test stops downstream models** in `dbt build` (this is the point)
- **Profile lookup**: CLI flag > env var > project dir > `~/.dbt/`
- **Staging shouldn't join** — joins belong in intermediate
- **`source()` takes TWO args**: source name, table name
- **`analyses/`** is NEVER run by `dbt run` — ad-hoc only
- **Ephemeral models** don't exist in the DB — can't query them in DuckDB CLI directly
- **`+model`** = upstream/ancestors · **`model+`** = downstream/descendants

That's it.

Print this. Carry it. Memorize it.

Then go drill the 50 practice questions.